

Getting Started with [moduleName] API

The [moduleName] API enables [frontEndName] with storage-as-a-service.

In this guide, you'll learn how to request an access token and make a simple API call.

1. Prerequisites

Complete the following items before continuing with this guide:

- Get a program to make requests, such as [Postman](#) (preferred) or [Insomnia](#).
- [Request access to the AD group](#) `resource.name`.

2. Create an OAuth Application

This section shows how to create an OAuth Application with the required scopes for the API authorization mechanisms.

1. Go to your [OAuth Apps](#)
2. Create a new OAuth app with the following custom scopes:
 - `resource.action::create:`
 - `resource.action::read:`
 - `resource.action::update:`
 - `resource.action::delete:`

3. Request an Access Token

This section shows how to generate an access token for the API.

3.1 [authenticationService] Domains

The [appName] leverages [authenticationService] credentials to authenticate requests.

Find the host URL based on the development environment of your choice:

[authenticationService] Domain	Host
Prod	<code>https://example.prod/v1/token</code>
QA	<code>https://example.qa/v1/token</code>
Dev	<code>https://example.dev/v1/token</code>

3.2 Query Parameters

The host URL accepts the following query parameters:

- `client` : Specifies the application client.
- `secret` : Specifies the application's environment secret.
- `type` : Specifies the authentication method.
- `scope` : Specifies the request method type for your access token.

3.2.1 Get client

1. Go to [OAuth Apps](#)

1. Go to [OAuth Applications](#)
2. Copy the value from the "client" column associated to the OAuth Application that you created on [step 2](#).
3. Paste the value on the `client` query parameter.

3.2.2 Get secret

1. Go to [AD Dashboard](#).
2. Under Applications, Select [appName]
3. Copy the value `secret`
4. Paste the value on the `secret` query parameter.

3.2.3 Get type

Enter "`client_credentials`".

3.2.4 Get scope

1. Go to [OAuth Applications](#).
2. Click on the OAuth Application that you created on [step 2](#).
3. Under **Custom Scopes**, copy one or more scopes for your access token.

NOTE

The authorization type variable locates at the end of the scope string, example:

```
resource.action::scope:
-----^-----
```

4. Paste the value on the `scope` query parameter.

3.3 Query the Host

This is an example to request a QA read access token:

POST

```
https://example.com/v1/token?client=[client]&secret=[secret]&type=client_credentials&scope=plat
```

This is the structure of the query:

- Host: `https://example.com.com`
- Endpoint: `/v1/token`
- Query Parameters: `?client=[client]&secret=[secret]&type=client_credentials&scope=app.global::read:`
- Content-Type: `application/x-www-form-urlencoded`

Replace the variables from the query parameters with the data from [step 3.2](#).

This is an example of the access token:

```
{
  "token_type": "Bearer",
  "expires_in": 3600,
  "access_token": {yourAccessToken},
  "scope": "{scope1} {scope2} ... {scopeN}"
}
```

To see more details about your access token, use a tool like jwt.io.

4. Stack Mapping

The [appName]] GraphQL API deploys in:

Stack	Backend URL	CDN URL
PROD		
UAT	<code>https://asd.example.com/graphql</code>	<code>https://uat.some.place/api/graphql</code>
STG	<code>https://qwe.example.com/graphql</code>	<code>https://stg.some.place/api/graphql</code>
QA	<code>https://wer.example.com/graphql</code>	<code>https://qa.some.place/api/graphql</code>
DEV	<code>https://sdf.example.com/graphql</code>	<code>https://dev.some.place/api/graphql</code>

NOTE

The URLs might change in newer release versions, if you're having issues visit the [Changelog](#).

5. Request Sample Query

Follow the steps to request a sample query:

1. Go to Postman or Insomnia
2. Create a new POST request
3. Enter a URL from the [Stack Mapping](#) table.
4. Enable bearer authentication and enter your `access_token`.
5. Enter the following data in Query:

```
mutation DeletePetMutation(  
  $petId: ID!  
  $requestedBy: String!  
  $confirmationCode: Int!  
) {  
  deletePet(  
    petId: $petId  
    requestedBy: $requestedBy  
    confirmationCode: $confirmationCode  
  )  
}
```

6. Enter the following Graphql variables:

```
{  
  "petId": "12345",  
  "requestedBy": "user@example.com",  
  "confirmationCode": 67890  
}
```

7. Submit the request, you must see a confirmation response:

Good work, you're ready to explore the [API reference](#).